Digital Systems Lab

Lecture 3

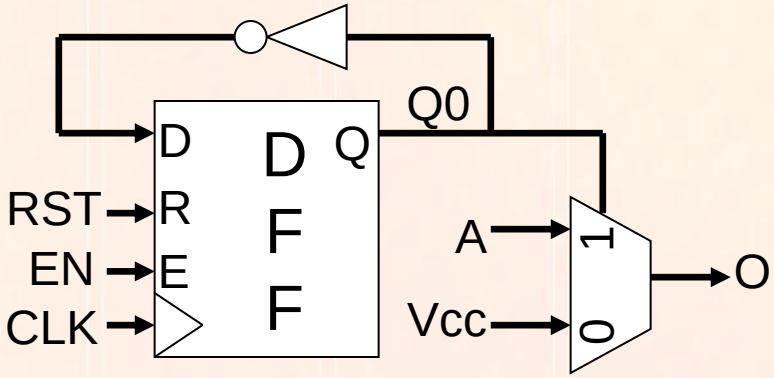
Electronic and Communication Engineering Department

Digital Systems Design Lab

Salam Kadhim

salam.alshemmari@uokufa.edu.iq

Sequential statements



- *All the VHDL constructs and statements seen so far (assignments, logic operators,* **`when/else`**, **`with/select`***) apply to combinational logic and execute concurrently.*
- *To implement sequential logic, however, concurrent execution is not appropriate – values change depending on control signals (e.g. the clock) in a timed conditional sequence.*
- *To impose a sequence in the execution of VHDL code, the instructions must be embedded into a process. A process is a region of VHDL code within an architecture that executes sequentially.*

Sequential statements

- *Conversely, all sequential statements must be executed within a process: this is necessary to implement sequential logic (latches, flip-flops, registers, counters, etc.)*

- *The syntax of a process declaration is the following:*

```
label: process (X,Y,…)
begin
        [statements]
end process label;
```

- *Processes are probably the VHDL constructs that seem most like software (unfortunately!)*

Sequential statements

```
label: process (X,Y,…)
begin
        [statements]
end process label;
```

- **label** *is a unique word that identifies the process (standard naming rules apply)*
- **(X,Y,…)** *is the (optional) sensitivity list, i.e., the list of signals that can (potentially) cause the output to change value: the statements in the process will be executed (in sequence) if and only if there is a change in one or more of the signals in the list.*
- *It is crucial to ensure that the sensitivity list is complete, i.e. that all signals that might affect the outputs are listed. If the list is not complete, the behaviour of the process will not be correct and synthesis will probably fail to generate the desired hardware. On the other hand, it should also be minimal, i.e. include only the signals that might affect the outputs.*

Sequential statements

Sequential statements can only be used within a process.

```
IF condition THEN
   -- sequential statements
ELSE
   -- sequential statements
END IF;
```

```
if (C0 = '1' and C1='0')
then
        O1 <= A;
        O2 <= B;
else
        O1 <= B;
        O2 <= A;
end if;
```

```
IF condition THEN
   -- sequential statements
ELSIF condition THEN
   -- sequential statements
ELSIF condition THEN
   -- sequential statements
ELSE
   -- sequential statements
END IF;
```

```
if (C0 = '1' and C1='0') then
        O <= A;
elsif (C0 = '1' and C1 = '1')
then
        O <= B;
elsif (C1 = '1') then
        O <= C;
else
        O <= D;
end if;
```

Sequential statements

Note that the first true condition causes its statements to be executed, after which execution will jump to the **`end if`** *clause.*

```
if x="0000" then
    z <= a;
else
    if x<="0101" then
        z <= b;
    else
        z <= c;
end if;
```

If **`x`** *has value* **`"0000"`**, *then both the first two conditions are true. Since* **`x="0000"`** *is tested first,* **`z`** *will be assigned value* **`a`**.

In a properly-written synthesizable process, all if-then-else clauses are nested

```
if x="0000" then
    z <= a;
if x<="0101" then
    z <= b;
 else
    z <= c;
end if;
```

BAD

Sequential statements

The **case** *statement takes a different branch depending on the value of a signal*

```
CASE object IS
   WHEN value_1 =>
       -- sequential statements
   WHEN value_2 =>
       -- sequential statements
   …
   WHEN OTHERS =>
       -- sequential statements
END CASE;
```

```
case S is
    when "00" =>
        O <= A;
    when "01" =>
        O <= B;
    when others =>
        null;
end case;
```

Catch-all condition (good coding practice)

The keyword **null** *is used when no action is to be taken*

Note that the values tested can be of different formats:

```
when "00" =>      -- specific binary

when 1 to 5 =>    -- range

when 6 | 8 =>     -- list
```

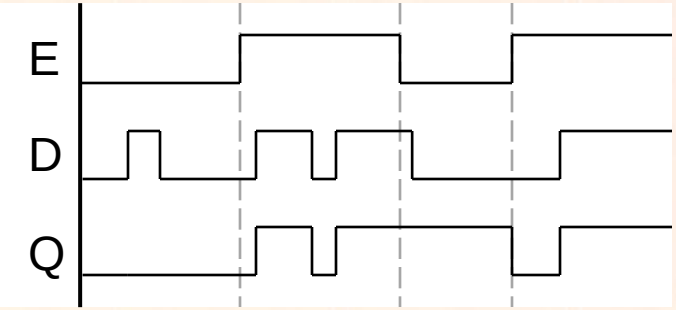
However, best to stick with specific binaries for the moment…

Combinational vs sequential logic

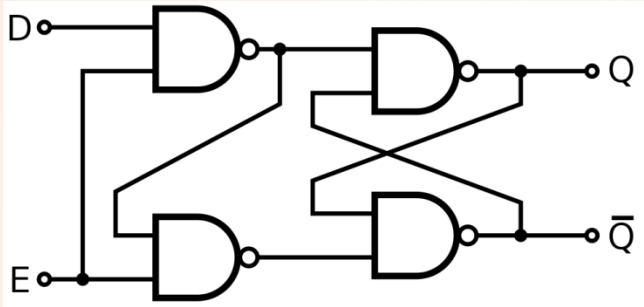
NEVER USE PROCESSES TO IMPLEMENT COMBINATIONAL LOGIC!

Example - Latch

- *First example of sequential component: the latch*



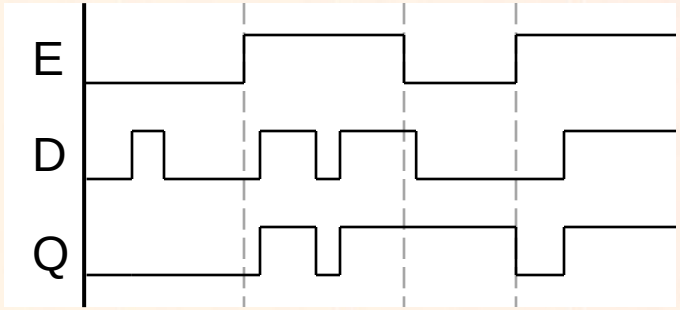
E	D	Action
0	0	keep state (Q <= Q)
0	1	keep state (Q <= Q)
1	0	set Q <= 0
1	1	set Q <=1



- *Latches (and indeed all sequential elements) could be described using logic gates with feedback, but this description is not convenient for most designs (and will probably cause massive warnings in synthesis)*

Example - Latch

- *First example of sequential component: the latch*



E	D	Action
0	0	keep state (Q <= Q)
0	1	keep state (Q <= Q)
1	0	set Q <= 0
1	1	set Q <=1

```
entity D_latch is
 port (E, D : in STD_LOGIC;
       Q: out STD_LOGIC);
end D_latch;

architecture arch of D_latch is
begin
  DLatch: process (E,D)
  begin
    if (E='1') then
       Q <= D;
    end if;
  end process DLatch;
end arch;
```

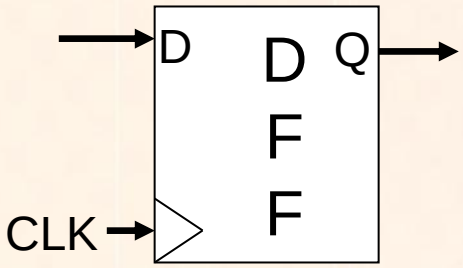
E,D in sensitivity list

Q <= Q is the default behaviour in a process (no need for "else" clause)

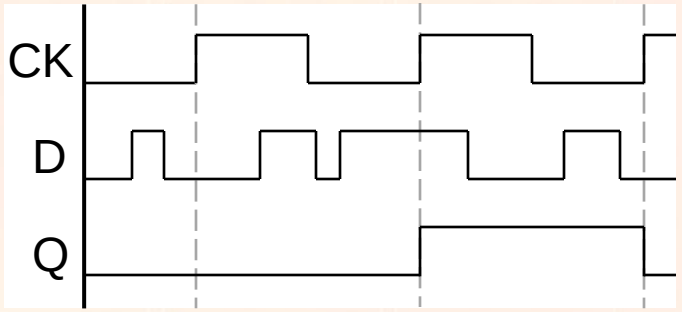
- *Latches are fairly common in logic design, because they are more compact than flip-flops… but in this module we will only use flip-flops (simpler designs)*

The clock signal

- *All sequential elements apart from latches are controlled by a clock signal.*



Clock	D	Q_{next}
Rising edge	0	0
Rising edge	1	1
Non-Rising	X	Q



Example: posedge triggered D-type FF

- *Clock distribution is one of the most challenging issues in modern circuit design – clocks should be handled with extreme care*
- *Standard, dedicated statements and constructs in VHDL are specifically designed to implement clock signals and should always be used.*

Design rules for FFs and Registers

- *The test on the clock can have different syntax depending on the test and on the libraries used.*
- *The most common structures are:*

```
element: process (CLK)
  begin
    if (CLK'event and CLK='1') then […]
```

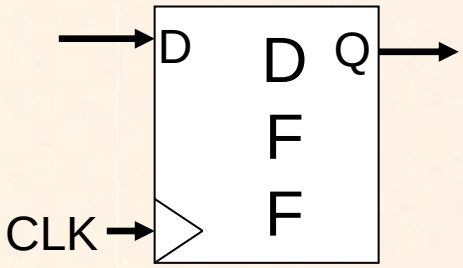
```
element: process
  begin
    if (rising_edge(clk)) then […]
```

Note dedicated syntax and functions

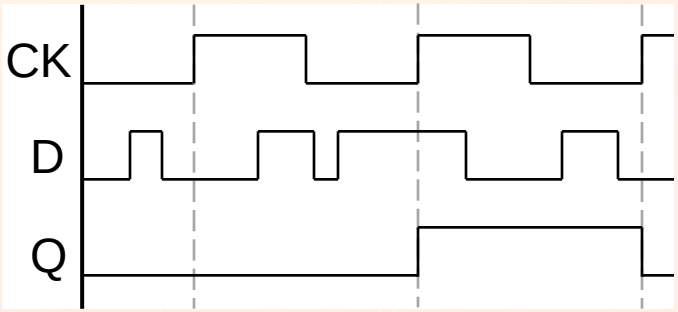
BEST

- *These are exactly equivalent (there are even a few more – see lecture 9)*

Example : D-type FF



Clock	D	Q_{next}
Rising edge	0	0
Rising edge	1	1
Non-Rising	X	Q



Example: posedge triggered D-type FF

```
entity posedgeD_FF is
 port (CLK, D : in STD_LOGIC;
       Q : out STD_LOGIC);
end posedgeD_FF;

architecture arch of posedgeD_FF is
begin
DFF: process (CLK)
  begin
     if (rising_edge(CLK)) then
        Q <= D;
     end if;
  end process DFF;
end arch;
```

Only CLK in sensitivity list

Q <= Q is the default behaviour in a process (no need for "else" clause)

The reset (clear) signal

- *Sequential elements are used to memorize a value*
- *When a circuit starts up, the value that is memorized in the elements is undetermined – this can affect the operation of the circuit: all sequential elements in a design should be initialized with a reset signal*
- *The initial, undetermined value is a correct representation of the initial state of the element and should not be "hidden" (even if VHDL allows you to do so by initializing signal values).*

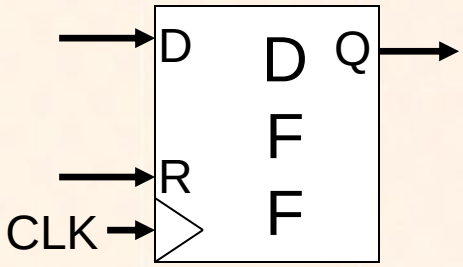
The reset (clear) signal

- *Of course, reset can also occur at any point in the operation of the circuit, not just at startup (e.g. in counters or finite state machines)*
- *The reset signal always overrides (has higher priority than) any other value assignment*
- *Reset signals can be synchronous (operate on the clock edge) or asynchronous (operate independently of the clock). In the latter case, prefer the use of the term clear*

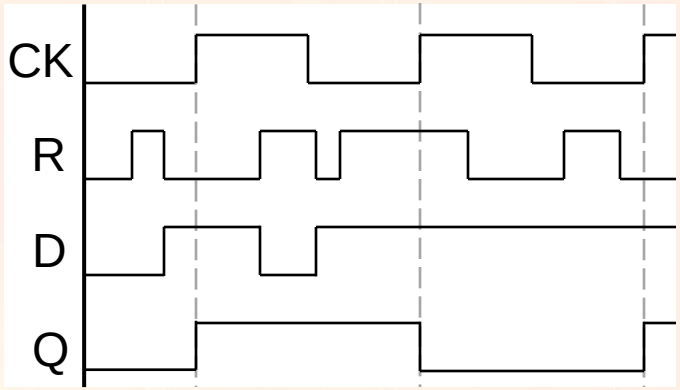
The reset (clear) signal

- *Synchronous reset*

GOOD DESIGN PRACTICE



Clock	D	R	Q_{next}
Rising edge	0	0	0
Rising edge	1	0	1
Rising edge	X	1	0
Non-Rising	X	X	Q



Example: posedge triggered D-type FF with synchronous reset

```
DFF: process (CLK)
  begin
    if (rising_edge(CLK)) then
      if (RST='1') then
            Q <= '0';
      else
            Q <= D;
      end if;
    end if;
  end process DFF;
```

Only CLK in sensitivity list

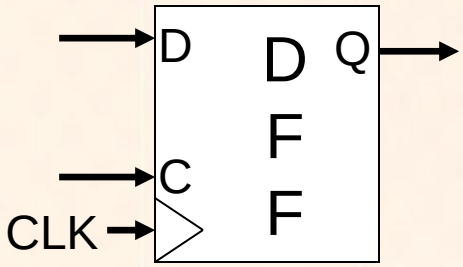
Reset tested immediately after clock

Reset value – usually (but not always) 0

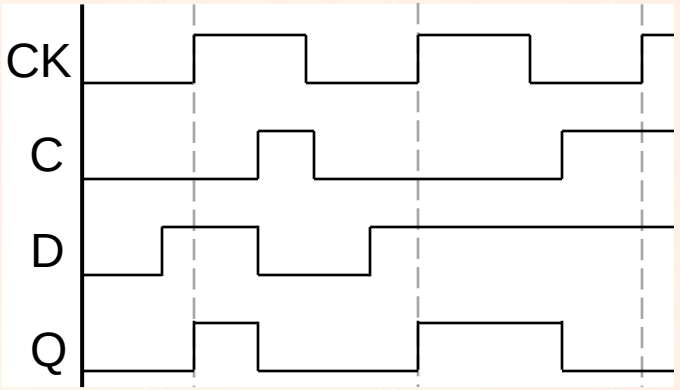
The reset (clear) signal

- *Asynchronous clear*

ONLY WHEN NECESSARY



Clock	D	C	Q_{next}
Rising edge	0	0	0
Rising edge	1	0	1
Non-Rising	X	0	Q
X	X	1	0



Example: posedge triggered D-type FF with asynchronous clear

```
DFF: process (CLK,CLR)
  begin
    if (CLR='1') then
       Q <= '0';
    else
        if (rising_edge(CLK)) then
          Q <= D;
       end if;
    end if;
  end process DFF;
```

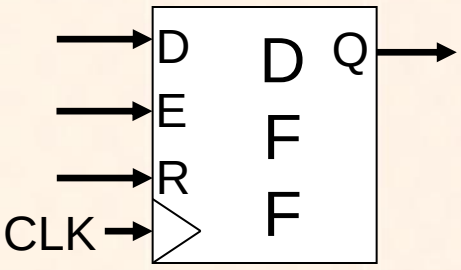
CLK and CLR in sensitivity list

Clear tested immediately before clock

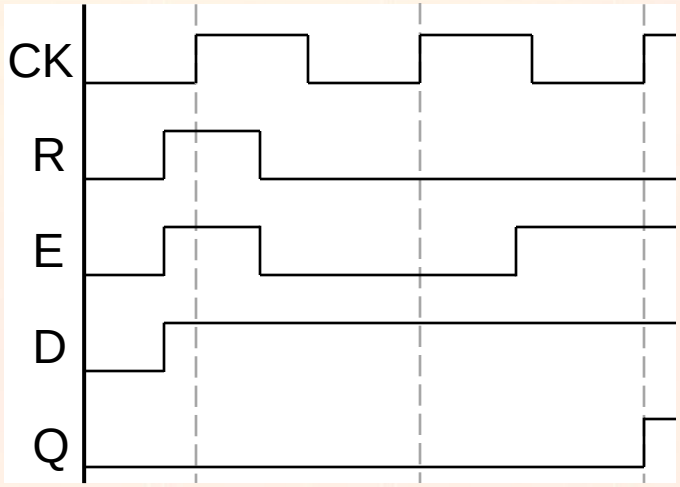
The enable signal

- *Clocks and clock distribution networks are one of the most sensitive areas of digital circuit design. Usually, complex devices rely on dedicated lines to distribute the clock signal to all parts of a circuit with minimum delay*
- *As a consequence, it is very bad design practice to apply any logic to a clock signal: all clocked elements in a design should operate directly on the clock signal*
- *To control the operation of a clocked element (for example, to make it memorize the input only at certain times), an enable signal should always be used instead of changing the clock*
- *Enable signals are always synchronous (the enable the clock!) and have lower priority compared to reset*

The enable signal



Clock	D	R	E	Q_n
Rising edge	0	0	1	0
Rising edge	1	0	1	1
Rising edge	X	0	0	Q
Rising edge	X	1	X	0
Non-Rising	X	X	X	Q



Example: posedge triggered D-type FF with synchronous reset and enable

```
DFF: process (CLK)
  begin
     if (rising_edge(CLK)) then
       if (RST='1') then
             Q <= '0';
       else
          if (EN = '1') then
             Q <= D;
          end if;
       end if;
    end if;
  end process DFF;
```

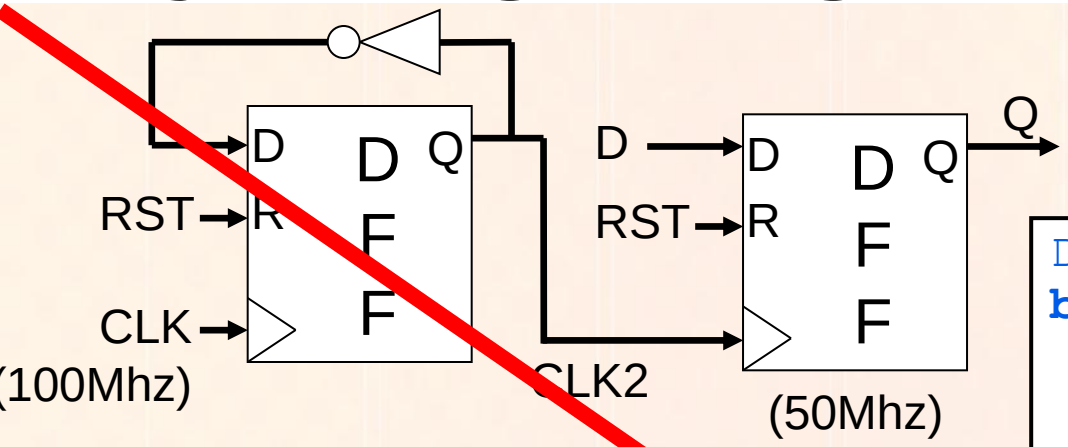
Only CLK in sensitivity list

Enable tested after clock and reset

Q <= Q is the default behaviour (no need for "else")

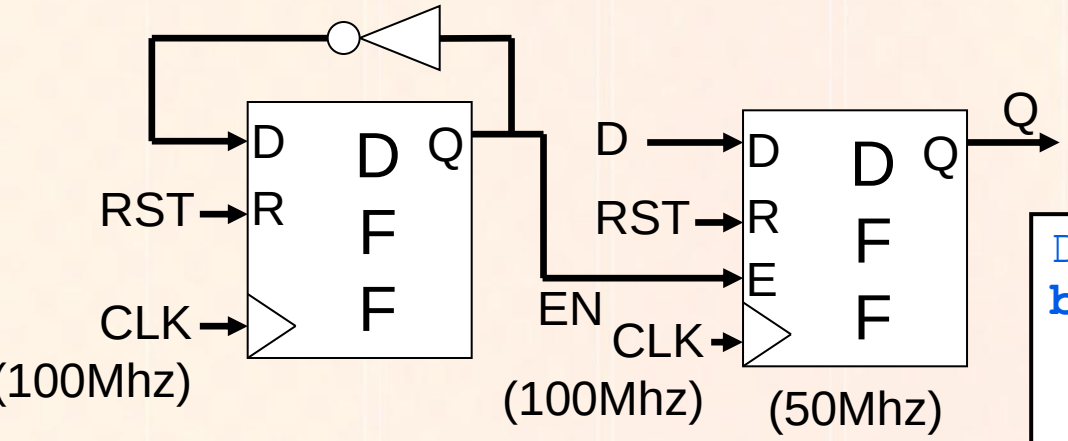
NOTE THAT ALL "IF" STATEMENTS ARE NESTED!

1st year digital logic




```
DFF1: process (CLK)
begin
  if (rising_edge(CLK)) then
    if (RST = '1') then
      CLK2 <= '0';
    else
      CLK2 <= not CLK2;
    end if;
  end if;
end process DFF;

DFF2: process (CLK2)
begin
  if (rising_edge(CLK2)) then
    if (RST = '1') then
      Q <= '0';
    elsif (EN = '1') then
      Q <= D;
    end if;
  end if;
end process DFF2;
```

Proper implementation



GOOD

```
DFF1: process (CLK)
begin
  if (rising_edge(CLK)) then
    if (RST = '1') then
      EN <= '0';
    else
      EN <= not EN;
    end if;
  end if;
end process DFF;

DFF2: process (CLK)
begin
  if (rising_edge(CLK)) then
    if (RST = '1') then
      Q <= '0';
    elsif (EN = '1') then
      Q <= D;
    end if;
  end if;
end process DFF2;
```

DFF with reset and enable

```
entity D_FF is
 port (CLK, Data_in, RST, EN : in STD_LOGIC;
       Q : out STD_LOGIC);
end D_FF;

architecture arch of D_FF is
begin
  DFF: process (CLK)
  begin
     if (rising_edge(CLK)) then
       if (RST = '1') then
         Q <= '0';
       elsif (EN = '1') then
         Q <= Data_in;
       end if;
    end if;
  end process DFF;
end arch;
```

Only clock in sensitivity list (fully synchronous process – nothing happens unless there is a rising edge in the clock)

DFF with clear and no enable

```
entity D_FF is
 port (CLK, Data_in, CLR : in STD_LOGIC;
       Q : out STD_LOGIC);
end D_FF;

architecture arch of D_FF is
begin
  DFF: process (CLK, CLR)
  begin
    if (CLR = '1') then
         Q <= '0';
    elsif (rising_edge(clk)) then
         Q <= Data_in;
    end if;
  end process DFF;
end arch;
```

Both clock and clear in sensitivity list (clear changes output even if there is no rising edge in the clock)

8-bit D-type register with reset and enable

```
entity REG_8BIT is
 port (CLK, RST, WEN : in STD_LOGIC;
       Data: in STD_LOGIC_VECTOR(7 downto 0);
       Q : out STD_LOGIC_VECTOR(7 downto 0));
end REG_8BIT;

architecture arch of REG_8BIT is
begin
  REG8: process (CLK)
  begin
     if (rising_edge(CLK)) then
       if (RST = '1') then
         Q <= (others => '0');
       elsif (WEN = '1') then
         Q <= Data;
       end if;
     end if;
  end process REG8;
end arch;
```

A multi-bit sequential component is identical to a single-bit one, except it is applied to vectors

More on this syntax later

Bidirectional 16-bit shift register with load

```
entity SREG_16BIT is
 port (CLK,LD,SH,DIR,RST: in STD_LOGIC;
       Data: in STD_LOGIC_VECTOR(15 downto 0);
       Q : out STD_LOGIC_VECTOR(15 downto 0));
end SREG_16BIT;

architecture arch of SREG_16BIT is
signal Qint : STD_LOGIC_VECTOR(15 downto 0);
begin
  SREG16: process
  begin
    if (rising_edge(CLK)) then
      if (RST = '1') then
        Qint <= (others => '0');
      elsif (LD = '1') then
        Qint <= Data;
      elsif (SH = '1') then
        if (DIR = '1') then
          Qint(15 downto 1) <= Qint(14 downto 0);
          Qint(0) <= '0';
        else
          Qint(14 downto 0) <= Qint(15 downto 1);
          Qint(15) <= '0';
        end if;
      end if;
    end if;
  end process SREG16;
  Q <= Qint;
end arch;
```

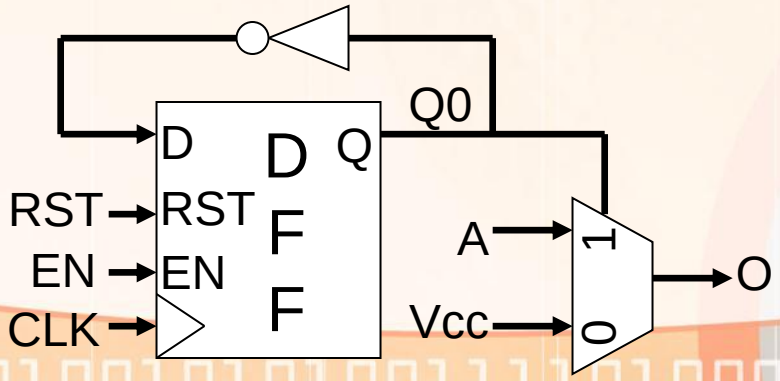
Why do we need this?

STILL ALL “IF” STATEMENTS ARE NESTED!

A note on coding style

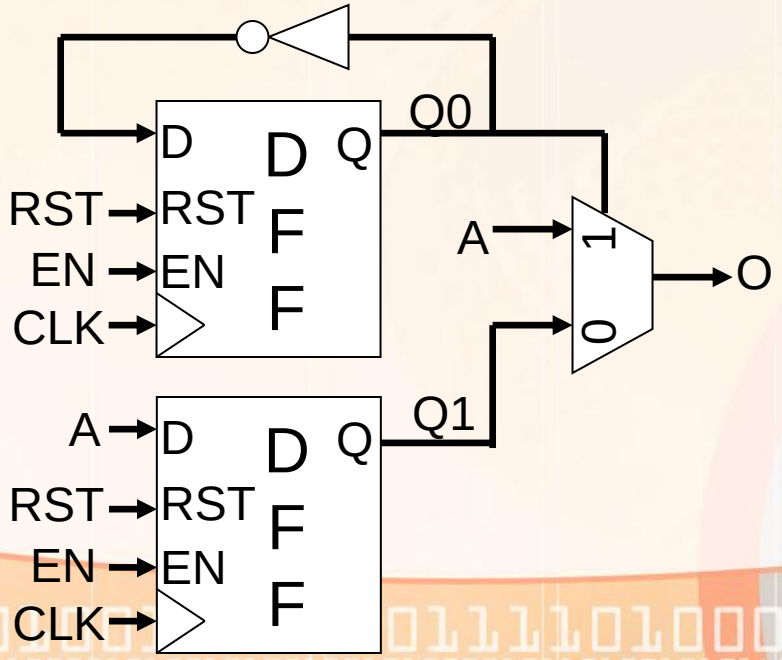
The examples you have seen are just that – examples. You don't need or normally want to use a complete entity for every sequential component – processes can be part of an architecture together with combinational logic

```
entity design is
 port (CLK, A, RST, EN : in STD_LOGIC;
       O : out STD_LOGIC);
end design;

architecture arch of design is
  signal Q : std_logic;
begin

  DFF: process (CLK)
  begin
    if (rising_edge(CLK)) then
       if (RST = '1') then
         Q <= '0';
       elsif (EN = '1') then
         Q <= not Q;
       end if;
    end if;
  end process DFF;

  O <= A when Q = '1' else '1';
end arch;
```



A note on coding style

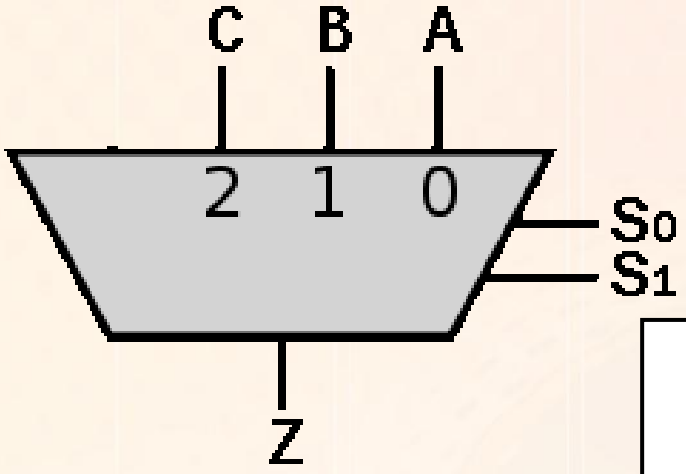
*If you implement multiple sequential elements in an architecture, you **can** use a single process – if all the elements have the same control signals (clock, reset, enable) – **BUT YOU DON'T HAVE TO!***




```
entity design is
 port (CLK, A, RST, EN : in STD_LOGIC;
       O : out STD_LOGIC);
end design;

architecture arch of design is
  signal Q0, Q1 : std_logic;
begin

  DFF: process (CLK)
  begin
    if (rising_edge(CLK)) then
       if (RST = '1') then
         Q0 <= '0';
         Q1 <= '1';
       elsif (EN = '1') then
         Q0 <= not Q0;
         Q1 <= A;
       end if;
    end if;
  end process DFF;

  O <= A when Q0 = '1' else Q1;
end arch;
```

Synthesizable VHDL - Processes

- *Good coding practice for synthesizable VHDL dictates that processes should be used only for sequential logic.*
- *This is not just a matter of style, but it helps avoid errors:*




```
bad_mux: PROCESS (a,b,c,s) IS
BEGIN
   IF (s(0) = '0') THEN
      IF (s(1) = '0') THEN
         o <= a;
      ELSE
         o <= b;
      END IF;
   ELSE
      IF (s(1) = '1') THEN
         o <= c;
      END IF;
   END IF;
END PROCESS;
```

Synthesizable VHDL - Processes

```
=========================================================================
*                      HDL Synthesis                                    *
=========================================================================

Performing bidirectional port resolution...

Synthesizing Unit <badmux>.
    Related source file is "C:/Xilinx_Designs/tests_for_modules/badmux.vhd".
WARNING:Xst:737 - Found 1-bit latch for signal <O>.
Unit <badmux> synthesized.


=========================================================================
HDL Synthesis Report

Macro Statistics
# Latches                                : 1
 1-bit latch                           : 1

=========================================================================
```

Synthesizable VHDL - Processes

- *This can be fixed easily:*

```
ok_mux: PROCESS (a,b,c,s) IS
BEGIN
   IF (s(0) = '0') THEN
      IF (s(1) = '0') THEN
         o <= a;
      ELSE
         o <= b;
      END IF;
   ELSE
      IF (s(1) = '1') THEN
         o <= c;
      ELSE
         o <= a;
      END IF;
   END IF;
END PROCESS;
```

- ***But multiplexers are COMBINATIONAL LOGIC:***

```
o <= a WHEN s = "00" ELSE
     b WHEN s = "10" ELSE
     c WHEN s = "11" ELSE
     a WHEN s = "01" ELSE
     'U';
```

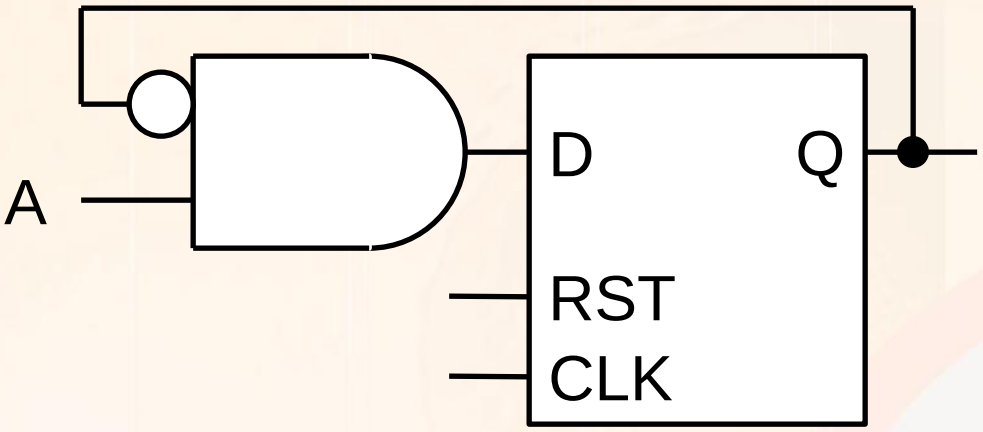
- ***or:***

```
WITH s SELECT
  o <= a WHEN "00",
       b WHEN "10",
       c WHEN "11",
       a WHEN "01",
       'U' WHEN OTHERS;
```

Combinational vs sequential logic

NEVER USE PROCESSES FOR COMBINATIONAL LOGIC!

This does not mean that there cannot be combinational logic inside a process:




```
DFF: process (CLK)
begin
  if (rising_edge(clk)) then
    if (RST = '1') then
      Q <= '0';
    elsif (EN = '1') then
      Q <= (not Q) and A;
    end if;
  end if;
end process DFF;
```

The rule is simple: <u>for any assignment inside a process, the left-hand side should always be the output of a sequential element</u> (the right hand side, however, is fair game...)